

Technische Dokumentation Kühlschrankmanager KM3003

Samuel Brunner

November 14, 2024

Contents

1	Einleitung	3
2	Plattform und Systemarchitektur	3
3	Software	4
3.1	Verwendete Bibliotheken und Frameworks	4
3.2	Datenbankstruktur	5
3.3	Testing	5
4	Deployment	6
5	Herausforderung bei der Implementierung	6
6	Ausblick und Verbesserungsvorschläge	6
7	Inline documentation anhängen	6

1 Einleitung

Der Kühltischmanager im Vertretungszimmer der Fachschaft ersetzt die Strichliste, welche früher am Kühltisch hing, und die Getränkekäufe der Vereinsmitglieder erfasste.

Früher wurden auf der Liste für jedes erworbene Getränk Striche in der Zeile des jeweiligen Mitglieds und in der Spalte des Getränks eingetragen. Regelmäßig zählte man die Striche, ermittelte so die Saldos der Mitglieder und erstellte eine neue Liste.

Der KM3003 ist die Weiterentwicklung der bisherigen Kühltischmanager (KM3000 bis KM3002). Der KM3002 basierte auf einem Raspberry Pi mit Touchscreen und NFC-Reader in einem 3D-gedruckten Gehäuse. Mitglieder wurden per Studentenausweis mit NFC identifiziert, und Käufe über den Touchscreen ausgewählt. Die Daten wurden in einer MySQL-Datenbank gespeichert, die jedoch nur negative Salden ermöglichte und so exakte Abrechnungen erzwang. Zudem gab es Verbesserungsbedarf in Codequalität, Zuverlässigkeit und Geschwindigkeit.

Der KM3003 erlaubt nun die Identifizierung von Mitgliedern und Produkten über das Scannen von Barcodes. Mitglieder werden über die Barcodes auf den Studentenausweisen erkannt, Produkte über EAN-Codes. Außerdem erlaubt er nun auch positive Saldos, sodass Mitglieder flexibel Guthaben einzahlen können.

Zusammengefasst ist der KM3003 ein Self-Checkout Point-of-Sale-Gerät mit Touchscreen und Barcode-Scanner.

2 Plattform und Systemarchitektur

Als Basis für den Kühltischmanager wurde ein Microsoft Surface 3 mit dem zugehörigen Dock verwendet. Das Surface 3 ist ein Tablet-Computer mit einem zuverlässigen kapazitiven Multi-Touchscreen und starker Treiberunterstützung unter Windows. Als Betriebssystem wird ein unverändertes Windows 10 Home verwendet, um eine gut getestete und stabile Basis zu bieten. Auch die Verfügbarkeit von Updates in der Zukunft wird so maximiert.

Weiterhin bieten die meisten Microsoft Surface Geräte einen Kioskmodus der spezielle Lösungen für Geräte mit festem und ortsunveränderlichem Verwendungszweck bietet. So optimiert der Kioskmodus das Ladeverhalten für Geräte, die permanent mit Strom versorgt werden, um die Akkulebensdauer

zu verlängern. Weiterhin kann der Zugang zu Anwendungen, die nicht dem gedachten Verwendungszweck entsprechen, beschränkt werden.

Zum scannen wird ein Freihandscanner (Modell DS9208) der Firma Zebra verwendet. Dieser wird per USB angeschlossen, meldet sich aber unter Windows, als ein über serielle Schnittstelle angebundenes Gerät. Dies erfolgt unter Verwendung des USB communications device class (USB CDC) Treibers.

Das Surface speichert keinerlei Daten. Hierfür wird eine externe MySQL-Datenbank verwendet, welche Nutzer, Produkte, Ein- bzw. Auszahlungen und Einkäufe enthält.

3 Software

Als Implementierungssprache wurde objektorientiertes Python3 verwendet. Python ist weit verbreitet und einfach zu lernen, was Wartungsarbeiten oder die Erweiterung mit neuen Features für zukünftige Mitglieder erleichtert.

Weiterhin können Python Anwendungen auf Linux und Windows Plattformen ausgeführt werden, sodass bei Bedarf auch das bisher verwendete Raspberry Pi mit externem Touchscreen weiter verwendet werden könnte.

Auch die Verfügbarkeit von bestehenden Bibliotheken für graphical user interface (GUI), Datenbankbindung und serielle Schnittstellen ist optimal.

3.1 Verwendete Bibliotheken und Frameworks

PySimpleGUI und Tkinter unterscheiden sich erheblich in Bezug auf Lizenzierung, Dokumentation und Architektur, was ihre Eignung für unterschiedliche Projekte beeinflusst.

Im Hinblick auf das Lizenzmodell erlaubt PySimpleGUI die kostenlose Nutzung primär für Hobby- und Bildungszwecke, sofern keine kommerzielle Absicht besteht. Für kommerzielle Nutzung oder kontinuierliche Updates fällt jedoch eine jährliche Gebühr von 99 US-Dollar an. Dieses Lizenzmodell kann einschränkend sein, falls sich ein Projekt in eine kommerzielle Richtung entwickelt, da nicht garantiert ist, dass die Lizenzbedingungen stabil bleiben. Tkinter hingegen ist unter der Tcl/Tk-Lizenz verfügbar, die seit 2010 unverändert ist und uneingeschränkte Rechte zur Modifikation, Verbreitung und kommerziellen Nutzung bietet. Dies macht Tkinter zu einer

verlässlicheren Wahl für langfristige Projekte, bei denen Stabilität der Lizenzierung entscheidend ist.

Ein weiterer Unterschied besteht in der Dokumentation. Die Dokumentation von PySimpleGUI ist stark beispielorientiert und richtet sich primär an Hobbyentwickler. Für erfahrene Entwickler, die eher eine formale API-Spezifikation als viele Beispiele bevorzugen, kann dies weniger hilfreich sein. Tkinter hingegen bietet eine professionellere Dokumentation mit detaillierten API-Spezifikationen, was die gezielte Nutzung einzelner Funktionen wie `configure()` erleichtert. Die Dokumentation von Tkinter ist daher besonders für Entwickler mit mehr Erfahrung besser geeignet.

Auch in der Architektur zeigen sich deutliche Unterschiede. Bei Tkinter wird jede Komponente als "Widget" innerhalb einer Hierarchie betrachtet und kann umfassend konfiguriert werden. Die Anordnung der Widgets wird von Geometriemanagern verwaltet, die eine feine Steuerung des Layouts und der Reaktionsfähigkeit ermöglichen. Diese ereignisgesteuerte Architektur erlaubt es der GUI, dynamisch auf Nutzereingaben zu reagieren, was Tkinter für Projekte mit komplexen, flexiblen Benutzeroberflächen besonders geeignet macht. PySimpleGUI hingegen arbeitet mit sogenannten "Elementen", die in Zeilen angeordnet und vertikal gestapelt werden. Zwar gibt es Container-Elemente zur Gruppierung, doch ist diese Layout-Struktur im Vergleich weniger flexibel und kann bei komplexen Designanforderungen einschränkend wirken.

Abschließend lässt sich sagen, dass Tkinter die bevorzugte Wahl für Projekte ist, die auf professionelle Dokumentation, stabile Lizenzierung und eine skalierbare Architektur angewiesen sind. Dank seiner stabilen Lizenz, der strukturierten Dokumentation und der vielseitigen Architektur eignet sich Tkinter besonders für langfristige und potenziell kommerzielle Projekte.

```
mysql connector  
this is a test
```

3.2 Datenbankstruktur

TODO an Normalformen angelehnt atomare Felder

3.3 Testing

Unit tests schreiben mit pyunit test Framework

4 Deployment

- deployment: surface, ansible for km3003 standalone cyclical restart windows task scheduler mysql db

5 Herausforderung bei der Implementierung

- wiederaufbau von Verbindungsabbrüchen (Datenbank und Scanner) zur Laufzeit

6 Ausblick und Verbesserungsvorschläge

UI Framework is kacke Sounds

7 Inline documetation anhängen

USB CDC USB communications device class

GUI graphical user interface

List of Figures

References

- [1] Microsoft Kiosk Mode <https://learn.microsoft.com/en-us/windows/iot/iot-enterprise/customize/kiosk-mode>
- [2] What is infrastructure as code (IaC)? <https://learn.microsoft.com/en-us/devops/deliver/what-is-infrastructure-as-code>
- [3] tkinter — Python interface to Tcl/Tk <https://docs.python.org/3/library/tkinter.html>
- [4] PyQt vs. Tkinter — Which Should You Choose for Your Next GUI Project? <https://www.pythonguis.com/faq/pyqt-vs-tkinter/>
- [5] PyQt6 Reference Guide <https://www.riverbankcomputing.com/static/Docs/PyQt6/index.html>